Technická príručka

Táto príručka slúži ako technická príručka k implementovanému dvojfaktorovému autentifikačnému systému klient–server, ktorý kombinuje autentifikáciu pomocou hesla, biometrickú verifikáciu a post-kvantové digitálne podpisy. Príručka je určená pre vývojára, administrátora alebo testera systému a popisuje architektúru riešenia, softvérové a hardvérové požiadavky, štruktúru projektu, postup inštalácie, jednotlivé scenáre spustenia a význam hlavných funkčných modulov.

Účel a zameranie príručky

Cieľom tejto technickej príručky je poskytnúť praktický a systematický návod na spustenie, konfiguráciu, pochopenie a údržbu implementovaného riešenia. Systém bol navrhnutý ako autentifikačný mechanizmus vhodný aj pre distribuované prostredia a scenáre, v ktorých vstavané zariadenie hostuje serverovú časť riešenia. Vychádza pritom z moderných kryptografických a bezpečnostných princípov.

Implementácia je rozdelená na tri hlavné časti:

- **klientská časť**, realizovaná ako webová aplikácia v prostredí React + TypeScript + Vite,
- **serverová časť**, implementovaná v Node.js + Express so SQLite databázou,
- **lokálny autentifikačný agent**, implementovaný v jazyku Python s využitím rámca Flask a komunikácie so snímačom odtlačku prsta AS608.

Systém podporuje dva základné scenáre prevádzky:

1. **Lokálny scenár**, v ktorom klientská časť, serverová časť aj agent fungujú v rámci jedného zariadenia alebo jedného pracovného prostredia.
2. **Distribuovaný scenár**, v ktorom serverová časť (prípadne aj statické súbory webového rozhrania) beží na zariadení Raspberry Pi 5 a autentifikačný agent je spustený na klientskom zariadení používateľa.

Charakteristika implementovaného riešenia

Autentifikačný systém pozostáva z dvoch logických fáz:

- **registrácia používateľa a zariadenia**,
- **prihlásenie používateľa s dvojfaktorovým overením**.

V prvej fáze sa realizuje heslová registrácia používateľa a následne registrácia zariadenia. Na klientskom zariadení sa pri tom vykoná registrácia biometrie, vygeneruje sa podpisový pár mldsa a súkromný kľúč sa uloží v zašifrovanej podobe.

V druhej fáze sa najprv dokončí heslová autentifikácia a až potom server vytvorí jednorazovú výzvu pre zariadením viazaný podpis. Lokálny agent po úspešnom biometrickom overení podpíše túto výzvu a server následne rozhodne o vydaní finálneho autentifikačného tokenu. Táto príručka sa preto sústreďuje už len na implementačný pohľad.

Architektúra systému

Implementácia je rozdelená medzi webového klienta, server a lokálneho agenta, pričom každý z týchto komponentov nesie odlišnú časť zodpovednosti. Ich stručné predstavenie uľahčuje orientáciu v zdrojových súboroch aj v neskoršom opise tokov registrácie a prihlásenia.

Klientská časť

Klientská časť je realizovaná ako jednostránková webová aplikácia. Implementované sú najmä tieto súbory:

- `client/src/pages/Login.tsx`,
- `client/src/pages/Dashboard.tsx`,
- `client/src/components/ProtectedRoute.tsx`,
- `client/src/App.tsx`.

Klientská časť zabezpečuje:

- zadanie používateľského mena a hesla,
- inicializáciu registrácie pomocou OPAQUE,
- inicializáciu prihlasovania pomocou OPAQUE,
- vyžiadanie registrácie zariadenia,
- komunikáciu s lokálnym agentom bežiacim na adrese `https://127.0.0.1:5555`,
- odoslanie podpisovej odpovede na server,
- uloženie tokenu po úspešnom prihlásení.

Serverová časť

Server je implementovaný v súbore `server/src/server.ts`. Zabezpečuje:

- obsluhu rest api koncových bodov,
- inicializáciu a správu OPAQUE serverového nastavenia,
- perzistenciu používateľských údajov do SQLite databázy `users.db`,
- správu výziev pre pqc podpis,
- overovanie podpisu pomocou verifikačného skriptu v jazyku Python,
- generovanie tokenov jwt po úspešnej autentifikácii,
- evidenciu metadát a výkonových metrík.

Lokálny autentifikačný agent

Agent je implementovaný najmä v súbore `tools/fingerprint/agent.py`. Jeho úlohou je:

- komunikovať so snímačom odtlačku prsta AS608 cez sériový port,
- realizovať biometrickú registráciu a verifikáciu,
- generovať podpisový pár mldsa,
- šifrovať a dešifrovať súkromný kľúč,
- podpisovať serverovú výzvu po úspešnej biometrickej verifikácii,
- spravovať lokálne pomocné dáta, počítadlo podpisov a logy metrík.

Štruktúra projektu

Odporúčaná pracovná štruktúra projektu je nasledujúca:

```
client/
  src/
  package.json
  package-lock.json
  tsconfig.json
  vite.config.ts

server/
  src/
    server.ts
    metrics.ts
  package.json
  package-lock.json
  tsconfig.json
  tls/
  users.db
  server_setup.txt

tools/
  pqc_verify.py
  metrics_export_csv.py
  fingerprint/
    agent.py
    as608_ops.py
    bio_register.py
    bio_verify.py
    crop.py
    image_norm.py
    live_finger.py
    metrics.py
    orientation.py
    requirements.txt
    tls/
```

```
pqc_keys/
fuzzy_extractor/
```

Pri čistej distribúcii zdrojových kódov sa odporúča neuchovávať priečinky `node_modules`, `.venv`, `__pycache__`, artefakty zostavenia a dočasné logy, pretože ide o generované súbory viazané na konkrétne prostredie.

Softvérové a hardvérové požiadavky

Pred samotným spustením je potrebné vymedziť prostredie, v ktorom bol prototyp navrhnutý, implementovaný a overovaný. Nasledujúce podsekcie preto osobitne uvádzajú softvérové aj hardvérové predpoklady jeho korektnej prevádzky.

Softvérové požiadavky

Pre korektné spustenie systému sa odporúča nasledovné prostredie:

- **Node.js** vo verzii 18 alebo novšej,
- **npm** ako správca balíkov pre klientskú časť a serverovú časť,
- **Python** vo verzii 3.10 alebo 3.11,
- **SQLite** ako lokálna databáza servera,
- operačný systém **Windows** alebo **Linux/Raspberry Pi OS**.

Hardvérové požiadavky

Hardvérová konfigurácia závisí od scenára nasadenia:

- počítač alebo notebook pre lokálny scenár,
- zariadenie **Raspberry Pi 5** pre distribuovaný scenár,
- snímač odtlačku prsta **AS608**,
- dostupný sériový port snímača, napríklad `COM3` vo Windows alebo `/dev/ttyUSB0` v Linuxe.

Závislosti projektu

Keďže jednotlivé časti riešenia využívajú odlišné technologické prostredia, je vhodné uviesť ich závislosti oddelene pre Node.js časť a pre Python agenta.

Klientská časť a serverová časť

Klientská časť aj serverová časť používajú samostatné Node.js závislosti definované v súboroch `package.json`. Inštalujú sa v príslušných priečinkoch príkazom:

```
npm install
```

Python agent

Python agent používa samostatný súbor `requirements.txt`. Odporúčaný obsah tohto súboru je:

```
Flask>=3.1,<3.2
flask-cors>=5.0,<6.0
requests>=2.31,<3.0
urllib3>=2.2,<3.0
pycryptodome>=3.20,<4.0
pqcrypto>=0.3,<0.4
pyserial>=3.5,<4.0
psutil>=5.9,<6.0
numpy>=1.26,<3.0
Pillow>=10.0,<12.0
opencv-python-headless>=4.10,<5.0
scikit-image>=0.24,<0.26
fuzzy-extractor==0.3
```

Ak sa vyžaduje grafické rozhranie knižnice OpenCV, je možné namiesto balíka `open cv-python-headless` použiť `opencv-python`. V typickom serverovom alebo vstavanom scenári je však vhodnejší variant bez grafického rozhrania.

Inštalácia systému

Keď sú známe požiadavky aj potrebné závislosti, možno prejsť k samotnej príprave prostredia. Inštalačný postup je preto rozdelený podľa jednotlivých častí riešenia tak, aby bolo možné klienta, server aj lokálneho agenta pripraviť reprodukovateľným spôsobom.

Klientská časť

V priečinku `client` je potrebné vykonať:

```
cd client
npm install
```

Na vývojové spustenie sa použije:

```
npm run dev
```

Pre produkčný alebo distribuovaný scenár sa vytvorí zostavenie:

```
npm run build
```

Serverová časť

V priečinku `server` je potrebné vykonať:

```
cd server
npm install
```

V aktuálnej implementácii je vývojový skript nakonfigurovaný predovšetkým pre Windows. Spustenie sa vykoná príkazom:

```
npm run dev
```

Pri nasadení na Linux alebo Raspberry Pi je vhodné explicitne nastaviť certifikačnú autoritu a následne spustiť server ručne, napríklad:

```
export NODE_EXTRA_CA_CERTS=./tls/rootCA.pem
npx tsx src/server.ts
```

Python agent

V priečinku `tools/fingerprint` je potrebné vytvoriť virtuálne prostredie a nainštalovať závislosti.

Vo Windows:

```
cd tools/fingerprint
python -m venv .venv
.venv\Scripts\activate
pip install -r requirements.txt
python agent.py
```

V Linuxe alebo na Raspberry Pi:

```
cd tools/fingerprint
python3 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
python agent.py
```

Agent štandardne počúva na adrese `127.0.0.1` a porte `5555` a používa tls certifikáty z priečinka `tls/`.

Scenáre spustenia

Spôsob spustenia systému sa prirodzene mení podľa toho, či všetky komponenty bežia na jednom zariadení, alebo či je server oddelený na samostatnom uzle. Preto sú nižšie zhrnuté oba podporované prevádzkové scenáre.

Lokálny scenár

V lokálnom scenári sú spustené tri časti systému:

1. klientská časť v priečinku `client`,
2. serverová časť v priečinku `server`,
3. agent v priečinku `tools/fingerprint`.

Odporúčaný postup:

1. spustiť klientskú časť pomocou `npm run dev`,
2. spustiť serverovú časť pomocou `npm run dev`,
3. aktivovať Python virtuálne prostredie a spustiť `python agent.py`,
4. v rozhraní klienta zadať používateľské meno, heslo a port snímača,

5. vykonať registráciu používateľa,
6. následne vykonať registráciu zariadenia,
7. po úspešnej registrácii sa prihlásiť.

Distribuovaný scenár

V distribuovanom scenári beží serverová časť na Raspberry Pi 5 a agent zostáva na klientskom počítači. Webové rozhranie je používateľovi sprístupnené z jeho bežného klientského zariadenia, na ktorom má k dispozícii prehliadač, periférie a biometrický snímač. Typický postup je:

1. na Raspberry Pi pripraviť a spustiť serverovú časť, prípadne naservovať aj statické zostavenie webového rozhrania,
2. na klientskom zariadení spustiť Python agenta,
3. z klientského zariadenia otvoriť webové rozhranie servera v prehliadači,
4. registráciu a prihlásenie vykonávať z klientského zariadenia,
5. komunikáciu so snímačom a podpisovanie realizovať cez lokálny agent.

Tento režim simuluje realistickejšie nasadenie, v ktorom centrálna serverová časť beží na vstavanom zariadení a biometrická verifikácia zostáva lokálna na klientskom zariadení.

Priebeh registrácie používateľa a zariadenia

Na lepšie pochopenie implementácie je užitočné sledovať registračný postup po jednotlivých krokoch, v poradí, v akom na seba nadväzujú požiadavky klienta, servera a lokálneho agenta.

Krok 1: OPAQUE – začiatok registrácie

Klient iniciuje registráciu používateľa zavolaním:

```
POST /api/register/start
```

Klientská časť vygeneruje `registrationRequest` pomocou OPAQUE knižnice a odošle ho serveru spolu s menom používateľa.

Server:

- vytvorí `registrationResponse`,
- vytvorí používateľský identifikátor `uid:<username>`,
- pripraví databázový záznam používateľa.

Krok 2: OPAQUE – ukončenie registrácie

Klientská časť dokončí registračný proces a serveru odošle:

`POST /api/register/finish`

Server:

- uloží `registrationRecord` do databázy,
- vygeneruje krátkodobý `regToken`,
- vráti informáciu, že používateľ môže pokračovať registráciou zariadenia.

Krok 3: Registrácia zariadenia, biometrie a PQC kľúčov

Po registrácii hesla nasleduje registrácia zariadenia cez agenta. Klientská časť iniciuje voči lokálnemu agentovi požiadavku:

`POST /pqc/register`

Agent následne:

1. vyžiada biometrickú registráciu na snímači AS608,
2. vytvorí mapovanie používateľa na identifikátor biometrickej šablóny,
3. vygeneruje pár verejného a súkromného kľúča algoritmu mldsa,
4. vytvorí náhodný kľúč kek,
5. zašifruje súkromný podpisový kľúč pomocou kek,
6. zašifruje kek pomocou hlavného kľúča zariadenia,
7. uloží verejný kľúč, zašifrovaný súkromný kľúč a zašifrovaný kek do priečinka `tools/fingerprint/pqc_keys/`,
8. vykoná lokálny test správnosti dešifrovania,
9. odošle verejný kľúč na server cez koncový bod `/api/pqc/register`.

Priebeh prihlásenia používateľa

Aj prihlasovací tok je vhodné čítať po jednotlivých krokoch, pretože práve tak sa najlepšie ukáže prechod od heslovej fázy k biometricky podmienenému podpisu serverovej výzvy.

Krok 1: OPAQUE – začiatok prihlásenia

Klient inicializuje prihlasovanie zavolaním:

`POST /api/login/start`

Server:

- načíta používateľa a jeho `registrationRecord`,
- vytvorí `loginResponse`,
- uloží dočasný stav prihlasovania do mapy `loginStates`.

Krok 2: OPAQUE – ukončenie prihlásenia

Klient dokončí OPAQUE výmenu a serveru odošle:

`POST /api/login/finish`

Server:

- dokončí serverovú časť OPAQUE protokolu,
- získa `sessionKey`,
- odstráni dočasný stav prihlásenia,
- vydá dočasný token `pre2faToken` s oprávnením `pqc_2fa`.

Krok 3: Vydanie výzvy

Po úspešnom overení hesla server vygeneruje výzvu:

`POST /api/pqc/challenge`

Server vytvorí:

- náhodnú 32-bajtovú výzvu,
- unikátne `challengeId`,
- čas expirácie,
- databázový záznam pre ochranu proti opakovanému prehratiu.

Krok 4: Biometrická verifikácia a podpis výzvy

Klientská časť odošle výzvu lokálnemu agentovi cez:

`POST /pqc/sign`

Agent:

1. dekóduje výzvu zo Base64 formátu,
2. načíta biometrickú šablónu používateľa,
3. vyzve používateľa na priloženie prsta,
4. overí zhodu odtlačku s uloženou šablónou na snímači,
5. načíta zašifrovaný kek a zašifrovaný súkromný kľúč,
6. dešifruje kek pomocou hlavného kľúča zariadenia,
7. dešifruje súkromný podpisový kľúč pomocou kek,
8. inkrementuje lokálne počítadlo podpisov,
9. zostaví podpisovaný dátový obsah,
10. dátový obsah kanonizuje a vytvorí jeho sha256 hash,

11. hash podpíše pomocou mldsa.

Výsledkom je objekt obsahujúci:

- `payload`,
- `signature`,
- `algorithm`,
- `fpScore`,
- `templateId`.

Krok 5: Overenie podpisu na serveri

Server prijme výsledok cez:

`POST /api/pqc/verify`

Server vykoná:

1. kontrolu existencie výzvy,
2. kontrolu, či výzva nebola použitá a neexpirovala,
3. kontrolu zhody používateľa a výzvy,
4. kontrolu, že používateľská verifikácia (`uv`) má hodnotu `true`,
5. kontrolu rastúceho počítadla podpisov,
6. kanonizáciu dátového obsahu,
7. výpočet hash hodnoty,
8. overenie podpisu pomocou uloženého verejného pqc kľúča.

Ak je overenie úspešné:

- výzva sa označí ako použitá,
- aktualizuje sa posledná hodnota podpisového počítadla,
- vytvorí sa finálny jwt,
- používateľ je považovaný za plne autentifikovaného.

Prehľad najdôležitejších koncových bodov

Pre praktickú orientáciu v implementácii je užitočné mať na jednom mieste zhrnuté najdôležitejšie rozhrania, cez ktoré jednotlivé komponenty medzi sebou komunikujú.

Serverové koncové body

Koncový bod	Metóda	Význam
`/api/health`	GET	Jednoduchá kontrola dostupnosti servera.
`/api/register/start`	POST	Začiatok registrácie používateľa cez OPAQUE.
`/api/register/finish`	POST	Dokončenie registrácie používateľa cez OPAQUE.
`/api/pqc/register-device`	POST	Volanie servera na lokálneho agenta pri registrácii zariadenia.
`/api/pqc/register`	POST	Uloženie verejného pqc kľúča používateľa na server.
`/api/login/start`	POST	Začiatok prihlasovania cez OPAQUE.
`/api/login/finish`	POST	Dokončenie prihlasovania cez OPAQUE.
`/api/pqc/challenge`	POST	Vytvorenie výzvy pre druhý faktor.
`/api/pqc/verify`	POST	Overenie podpisu výzvy.
`/api/admin/users`	GET	Výpis používateľov uložených v databáze.
`/api/register/biometric`	POST	Uloženie pomocných identifikačných údajov biometrickej väzby (napr. `templateId`), nie obrazu odtlačku.
`/api/verify/biometric`	POST	Pomocné overenie biometrickej väzby nad odvodenými údajmi. Nejde o serverové spracovanie surového odtlačku.
`/api/biometric/helper/:username`	GET	Získanie pomocného identifikátora biometrickej väzby (napr. `templateId`).

Koncové body lokálneho agenta

Koncový bod	Metóda	Význam
`/health`	GET	Kontrola dostupnosti agenta.
`/pqc/register`	POST	Biometrická registrácia a generovanie pqc kľúčov.
`/pqc/sign`	POST	Biometrická verifikácia a podpis výzvy.

Popis hlavných zdrojových súborov

Z veľkého počtu projektových súborov majú pre hlavný autentifikačný tok rozhodujúci význam najmä nasledujúce moduly.

client/src/pages/Login.tsx

Tento súbor implementuje hlavné používateľské rozhranie systému. Obsahuje tri hlavné pracovné toky:

- `registerFlow()` – registrácia používateľa cez OPAQUE,
- `registerDeviceFlow()` – registrácia zariadenia cez lokálneho agenta,
- `loginFlow()` – prihlásenie s OPAQUE a následným pqc podpisom výzvy.

Súčasťou formulára je aj pole pre zadanie portu snímača, napríklad `COM3`.

server/src/server.ts

Ide o hlavný serverový súbor. Realizuje:

- inicializáciu Express aplikácie,
- načítanie alebo generovanie OPAQUE serverového nastavenia,
- migráciu SQLite databázy,
- vydávanie tokenov jwt s obmedzeným rozsahom oprávnení,
- správu prihlasovacích stavov,
- správu výzvy a ochranu proti opakovanému prehratiu,
- komunikáciu s verifikačným skriptom v jazyku Python `tools/pqc_verify.py`.

server/src/metrics.ts

Tento modul implementuje merací mechanizmus servera. Obsahuje medzivrstvu a pomocné funkcie na vytváranie meracích úsekov, zachytávanie trvania operácií, väzbu na `trace_id` a export diagnostických údajov.

tools/fingerprint/agent.py

Predstavuje centrálny riadiaci súbor lokálneho agenta. Implementuje:

- server Flask,
- obsluhu biometrickej registrácie a verifikácie,
- generovanie pqc kľúčov,
- šifrovanie a dešifrovanie súkromného kľúča,
- správu hlavného kľúča zariadenia,
- vedenie podpisového počítadla,
- komunikáciu so serverom pri registrácii verejného kľúča.

`tools/fingerprint/as608_ops.py`

Obsahuje nízkoúrovňové operácie pre komunikáciu so snímačom AS608, vrátane registrácie biometrie a verifikácie šablón odtlačku prsta.

`tools/pqc_verify.py`

Tento skript slúži na overenie pqc podpisu zo serverovej strany. Server ho volá ako externý Python proces a vstupné dáta mu odovzdáva cez štandardný vstup vo formáte json.

Perzistentné dáta a ich význam

Z bezpečnostného aj prevádzkového hľadiska je dôležité rozlišovať medzi údajmi, ktoré ostávajú na serveri, a údajmi viazanými na konkrétne klientske zariadenie.

Databáza servera

SQLite databáza `users.db` obsahuje:

- používateľské meno,
- OPAQUE `userIdentifier`,
- `registrationRecord`,
- pomocné biometrické údaje,
- verejný pqc kľúč,
- názov použitého algoritmu,
- čas vytvorenia pqc kľúča,
- poslednú známu hodnotu podpisového počítadla.

Druhá tabuľka obsahuje výzvy pre druhý faktor a slúži na ochranu proti opakovanému prehratiu požiadaviek.

Lokálne dáta agenta

V priečinku `tools/fingerprint/pqc_keys/` sa nachádzajú najmä:

- verejné kľúče používateľov,
- zašifrované súkromné kľúče,
- zašifrované kek kľúče,
- hlavný kľúč zariadenia,
- mapovanie identifikátorov biometrických šablón,
- súbor `sign_counter.json`.

Tieto dáta sú závislé od behu systému a pri čistej distribúcii projektu sa neodporúča ich archivovať ako súčasť zdrojových kódov. Z biometrického hľadiska pritom nejde o uloženie obrazu odtlačku prsta, ale iba o mapovanie na `templateId` a prípadné pomocné odvodené údaje potrebné pre lokálnu prácu so snímačom. Z týchto údajov nie je možné spätne rekonštruovať odtlačok.

Bezpečnostné vlastnosti systému

Implementácia využíva viacero bezpečnostných mechanizmov:

- heslová vrstva je realizovaná cez OPAQUE,
- súkromný podpisový kľúč nie je ukladaný v otvorenej podobe,
- biometria neslúži priamo ako podpisový kľúč, ale ako ochranný faktor pri jeho použití,
- výzva je časovo obmedzená a jednorazová,
- podpisové počítadlo znižuje riziko útokov opakovaným prehratím,
- tokeny s obmedzeným rozsahom oprávnení oddeľujú registračné a prihlasovacie oprávnenia,
- systém uchováva metriky a identifikátory trasovania pre audit a analýzu správania.

Meranie výkonu a logovanie

Súčasťou implementácie je vlastný modul metrík. Každá významná operácia je obalená meracím úsekom, v rámci ktorého sa zaznamenávajú:

- fáza operácie,
- názov operácie,
- identifikátor trasovania,
- čas trvania,
- voliteľné doplnkové metadáta.

Takto je možné samostatne analyzovať:

- OPAQUE registráciu,
- OPAQUE prihlásenie,
- generovanie výzvy,
- registráciu odtlačku prsta,
- biometrickú verifikáciu,
- generovanie pqc kľúčov,

- podpisovanie výzvy,
- overovanie podpisu,
- databázové a http operácie.

Logy sú následne ukladané do jsonl súborov a môžu byť exportované do csv pre ďalšie spracovanie v kontingenčných tabuľkách a grafoch.

Odporúčania pre čistenie a údržbu projektu

Pri príprave projektu na odovzdanie alebo nasadenie sa odporúča:

- odstrániť priečinky `node_modules/`,
- odstrániť Python virtuálne prostredie `.venv/`,
- odstrániť priečinky `__pycache__/`,
- odstrániť dočasné logy,
- nezdieľať produkčné alebo testovacie tls kľúče,
- nezdieľať reálnu databázu používateľov,
- nezdieľať lokálne pqc kľúče a biometrické mapovania.

Pre verzionovanie je vhodné do súboru `.gitignore` zaradiť najmä:

- `client/node_modules/`,
- `server/node_modules/`,
- `client/dist/`,
- `tools/fingerprint/.venv/`,
- `server/users.db`,
- `tools/fingerprint/pqc_keys/`,
- `server/tls/*.pem`,
- `tools/fingerprint/tls/*.pem`,
- `logs/`,
- `*.jsonl`.

Najčastejšie problémy a ich riešenia

Pri inštalácii a testovaní prototypu sa opakovane objavujú niektoré typické integračné a prevádzkové ťažkosti. Nasledujúce podsekcie zhŕňajú tie najbežnejšie spolu s odporúčaným spôsobom ich overenia.

Agent nie je dostupný

Ak klientská časť alebo server nevie kontaktovať agenta na adrese `https://127.0.0.1:5555`, treba overiť:

- či je agent spustený,
- či používa správny certifikát a kľúč,
- či lokálny firewall neblokuje spojenie,
- či prehliadač alebo klient akceptuje lokálny certifikát.

Snímač odtlačku prsta neodpovedá

V takom prípade je potrebné overiť:

- správnosť zadaného portu,
- fyzické pripojenie snímača,
- prístupové práva k sériovému portu,
- či port nie je obsadený inou aplikáciou.

Overenie PQC zlyhá

Ak overenie podpisu zlyhá, je potrebné skontrolovať:

- dostupnosť modulu `pqcrypto`,
- existenciu správneho interpretéra Pythonu pre serverové overovanie,
- zhodu verejného kľúča uloženého na serveri a kľúča používaného agentom,
- korektnosť kanonizácie dátového obsahu a hashovania.

Nepodarí sa spustiť server na Raspberry Pi

Treba overiť:

- že zostavenie klientskej časti bolo úspešne vytvorené,
- že bola správne nastavená premenná `NODE_EXTRA_CA_CERTS`,
- že použité cesty k certifikátom a interpretéru Pythonu zodpovedajú prostrediu Linuxu.

Zhrnutie

Implementovaný systém je praktickou realizáciou viacfaktorovej autentifikácie, v ktorej sa kombinuje bezpečné spracovanie hesla pomocou OPAQUE, lokálna biometrická verifikácia a post-kvantové digitálne podpisy mldsa. Technická príručka ukazuje, že riešenie je modulárne, rozšíriteľné a vhodné aj pre scenáre, v ktorých serverová časť beží na vstavanom zariadení a citlivé kryptografické operácie ostávajú viazané na klienta.

Z implementačného hľadiska je dôležité najmä korektné oddelenie klientskej časti, serverovej časti a lokálneho agenta, správna správa tls certifikátov, príprava Python prostredia podľa `requirements.txt` a dôsledná práca s lokálnymi kryptografickými a biometrickými dátami. Po splnení týchto predpokladov je systém možné spoľahlivo spustiť, testovať a ďalej rozvíjať.